

TOPIC 1: INTRODUCTION

1. Given an array of strings `words`, return the first palindromic string in the array. If there is no such string, return an empty string `""`. A string is palindromic if it reads the same forward and backward.

Example 1:

Input: `words = ["abc", "car", "ada", "racecar", "cool"]`

Output: `"ada"`

Explanation: The first string that is palindromic is `"ada"`.

Note that `"racecar"` is also palindromic, but it is not the first.

Example 2:

Input: `words = ["notapalindrome", "racecar"]`

Output: `"racecar"`

Explanation: The first and only string that is palindromic is `"racecar"`.

Aim

To write a Python program to find the **first palindromic string** in a given array of strings. If no palindrome exists, return an empty string `""`.

Algorithm

1. Start.
2. Define a function `firstPalindrome(words)`.
3. Traverse each word in the list.
4. Check whether the word is equal to its reverse.
5. If it is a palindrome, return the word.
6. If no palindrome is found, return an empty string `""`.
7. Stop.

Example 1

main.py	Output
<pre>1- def firstPalindrome(words): 2- for word in words: 3- if word == word[::-1]: 4- return word 5- return "" 6 7 words = ["abc", "car", "ada", "racecar", "cool"] 8 print(firstPalindrome(words))</pre>	<pre>ada === Code Execution Successful ===</pre>

Time Complexity

$O(n \times m)$

Where:

- n = Number of strings in the array
- m = Average length of each string

Space Complexity

$O(1)$

Example 2

main.py	Output
<pre>1- def firstPalindrome(words): 2- for word in words: 3- if word == word[::-1]: 4- return word 5- return "" 6 7 words = ["notapalindrome", "racecar"] 8 print(firstPalindrome(words))</pre>	<pre>racecar === Code Execution Successful ===</pre>

Time Complexity

$O(n \times m)$

Space Complexity

$O(1)$

2. You are given two integer arrays `nums1` and `nums2` of sizes `n` and `m`, respectively.

Calculate the following values: `answer1` : the number of indices `i` such that `nums1[i]` exists in `nums2`. `answer2` : the number of indices `i` such that `nums2[i]` exists in `nums1`. Return `[answer1, answer2]`.

Example 1:

Input: `nums1 = [2,3,2]`, `nums2 = [1,2]`

Output: `[2,1]`

Explanation:

Example 2:

Input: `nums1 = [4,3,2,3,1]`, `nums2 = [2,2,5,2,3,6]`

Output: `[3,4]`

Explanation:

Aim

To write a Python program to count:

- **answer1**: Number of elements in `nums1` that exist in `nums2`.
- **answer2**: Number of elements in `nums2` that exist in `nums1`.
Return the result as `[answer1, answer2]`.

Algorithm

1. Start.
2. Read the arrays `nums1` and `nums2`.
3. Convert both arrays into sets for faster searching.
4. Count the elements in `nums1` that are present in `nums2`.
5. Count the elements in `nums2` that are present in `nums1`.
6. Print the result as `[answer1, answer2]`.
7. Stop.

Example 1:

```
1 - def findIntersectionValues(nums1, nums2):
2     set1 = set(nums1)
3     set2 = set(nums2)
4
5     answer1 = 0
6     for x in nums1:
7         if x in set2:
8             answer1 += 1
9
10    answer2 = 0
11    for x in nums2:
12        if x in set1:
13            answer2 += 1
14
15    return [answer1, answer2]
16
17    nums1 = [2, 3, 2]
18    nums2 = [1, 2]
19
20    print(findIntersectionValues(nums1, nums2))
```

[2, 1]

Time Complexity

$O(n + m)$

Space Complexity

$O(n + m)$

Example 2

```
1 - def findIntersectionValues(nums1, nums2):
2     set1 = set(nums1)
3     set2 = set(nums2)
4
5     answer1 = 0
6     for x in nums1:
7         if x in set2:
8             answer1 += 1
9
10    answer2 = 0
11    for x in nums2:
12        if x in set1:
13            answer2 += 1
14
15    return [answer1, answer2]
16
17    nums1 = [4, 3, 2, 3, 1]
18    nums2 = [2, 2, 5, 2, 3, 6]
19
20    print(findIntersectionValues(nums1, nums2))
```

[3, 4]

Time Complexity

$O(n + m)$

Space Complexity

$O(n + m)$

3. You are given a 0-indexed integer array `nums`. The distinct count of a subarray of `nums` is defined as: Let `nums[i..j]` be a subarray of `nums` consisting of all the indices from `i` to `j` such that $0 \leq i \leq j < \text{nums.length}$. Then the number of distinct values in `nums[i..j]` is called the distinct count of `nums[i..j]`. Return the sum of the squares of distinct counts of all subarrays of `nums`. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,2,1]`

Output: 15

Explanation: Six possible subarrays are:

[1]: 1 distinct value

[2]: 1 distinct value

[1]: 1 distinct value

[1,2]: 2 distinct values

[2,1]: 2 distinct values

[1,2,1]: 2 distinct values

The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 + 2^2 + 2^2 + 2^2 = 15$.

Example 2:

Input: `nums = [1,1]`

Output: 3

Explanation: Three possible subarrays are:

[1]: 1 distinct value

[1]: 1 distinct value

[1,1]: 1 distinct value

The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 = 3$.

Aim

To write a Python program to find the **sum of the squares of the distinct counts of all subarrays** of a given array.

Algorithm

1. Start.
2. Read the array nums.
3. Initialize total = 0.
4. Traverse all possible starting indices.
5. For each starting index, create an empty set.
6. Traverse all ending indices from the starting index.
7. Add the current element to the set.
8. Find the number of distinct elements using the size of the set.
9. Add the square of the distinct count to total.
10. Print total.
11. Stop.

Example 1

```
1 def sumCounts(nums):
2     total = 0
3
4     for i in range(len(nums)):
5         s = set()
6         for j in range(i, len(nums)):
7             s.add(nums[j])
8             total += len(s) ** 2
9
10    return total
11
12    nums = [1, 2, 1]
13    print(sumCounts(nums))
```



15

Time Complexity

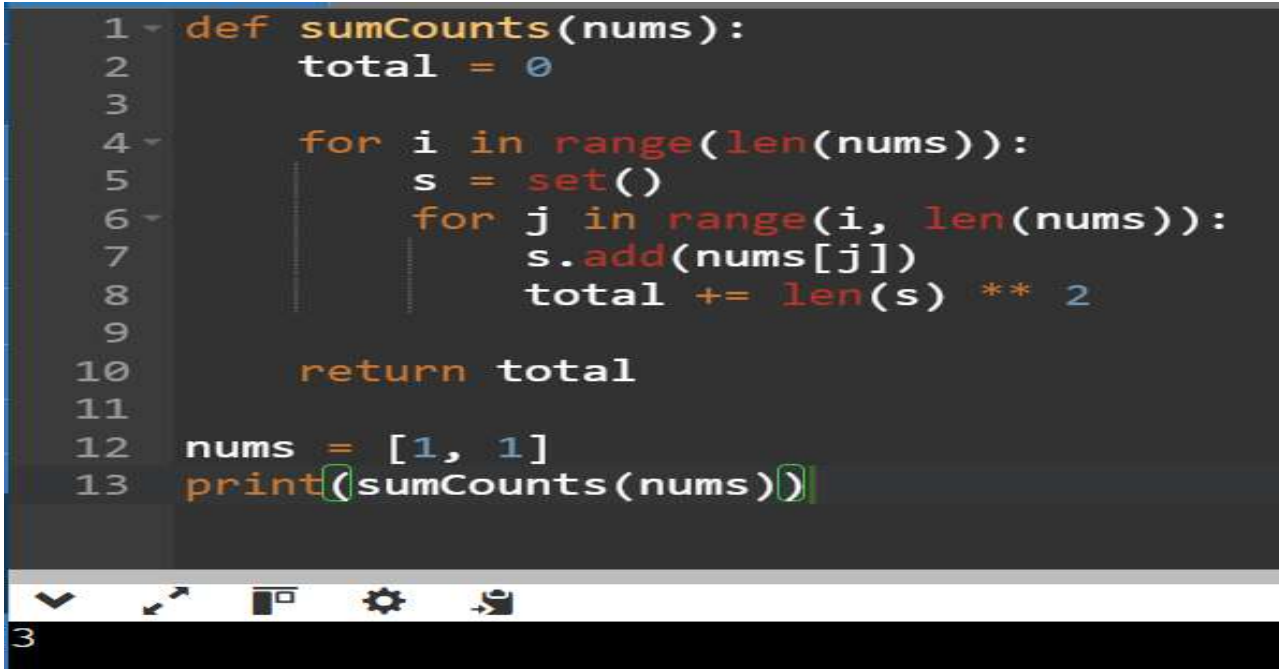
$O(n^2)$

Space Complexity

$O(n)$

Example 2

```
1 def sumCounts(nums):
2     total = 0
3
4     for i in range(len(nums)):
5         s = set()
6         for j in range(i, len(nums)):
7             s.add(nums[j])
8             total += len(s) ** 2
9
10    return total
11
12    nums = [1, 1]
13    print(sumCounts(nums))
```



3

Time Complexity

$O(n^2)$

Space Complexity

$O(n)$

4. Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return the number of pairs (i, j) where $0 \leq i < j < n$, such that `nums[i] == nums[j]` and $(i * j)$ is divisible by `k`.

Example 1:

Input: `nums = [3,1,2,2,2,1,3]`, `k = 2`

Output: 4

Explanation:

There are 4 pairs that meet all the requirements:

- `nums[0] == nums[6]`, and $0 * 6 == 0$, which is divisible by 2.
- `nums[2] == nums[3]`, and $2 * 3 == 6$, which is divisible by 2.
- `nums[2] == nums[4]`, and $2 * 4 == 8$, which is divisible by 2.
- `nums[3] == nums[4]`, and $3 * 4 == 12$, which is divisible by 2.

Example 2:

Input: `nums = [1,2,3,4]`, `k = 1`

Output: 0

Explanation: Since no value in `nums` is repeated, there are no pairs (i, j) that meet all the requirements.

Aim

To write a Python program to count the number of pairs (i, j) such that:

- $0 \leq i < j < n$
- `nums[i] == nums[j]`
- $(i * j)$ is divisible by `k`.

Algorithm

1. Start.
2. Read the array `nums` and integer `k`.
3. Initialize `count = 0`.
4. Traverse all pairs (i, j) where $i < j$.
5. Check if `nums[i] == nums[j]`.

6. Check if $(i * j) \% k == 0$.
7. If both conditions are true, increment count.
8. Print count.
9. Stop.

Example :1

```
1 def countPairs(nums, k):
2     count = 0
3     n = len(nums)
4
5     for i in range(n):
6         for j in range(i + 1, n):
7             if nums[i] == nums[j] and (i * j) % k == 0:
8                 count += 1
9
10    return count
11
12    nums = [3, 1, 2, 2, 2, 1, 3]
13    k = 2
14
15    print(countPairs(nums, k))
```

input

4

Time Complexity: $O(n^2)$

Space Complexity : $O(1)$

Example 2

```
1 def countPairs(nums, k):
2     count = 0
3     n = len(nums)
4
5     for i in range(n):
6         for j in range(i + 1, n):
7             if nums[i] == nums[j] and (i * j) % k == 0:
8                 count += 1
9
10    return count
11
12    nums = [1, 2, 3, 4]
13    k = 1
14
15    print(countPairs(nums, k))
```

input

0

Time Complexity : $O(n^2)$

Space Complexity : $O(1)$

5. Write a program FOR THE BELOW TEST CASES with least time complexity

Test Cases: -

Input: {1, 2, 3, 4, 5} Expected Output: 5

Input: {7, 7, 7, 7, 7} Expected Output: 7

Input: {-10, 2, 3, -4, 5} Expected Output: 5

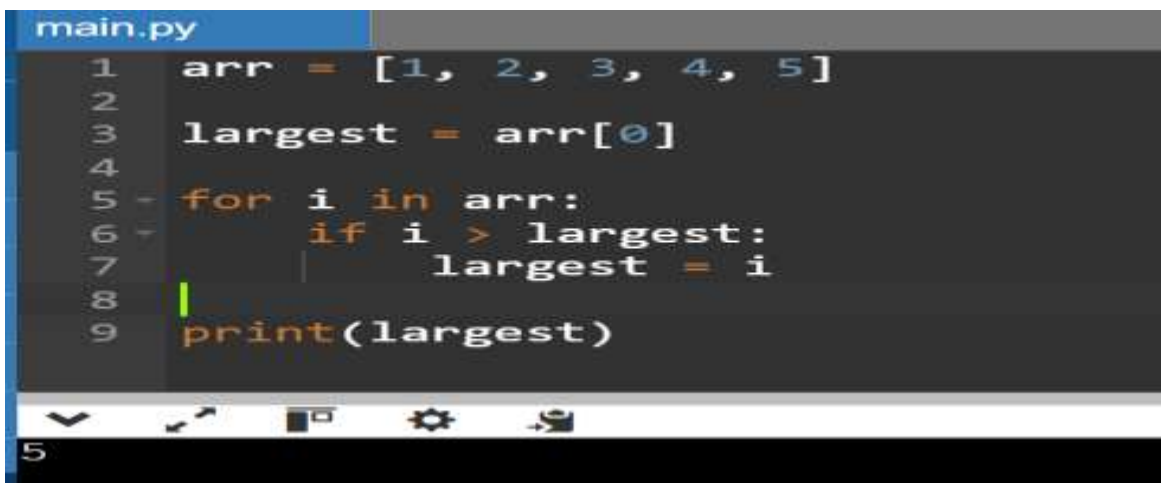
Aim

To write a Python program to find the **largest element** in a given array with the least time complexity.

Algorithm

1. Start.
2. Read the array.
3. Assume the first element is the largest.
4. Traverse the remaining elements.
5. If an element is greater than the current largest, update the largest.
6. Print the largest element.
7. Stop.

TEST:1

A screenshot of a Python IDE window titled 'main.py'. The code defines an array 'arr' with values [1, 2, 3, 4, 5], initializes 'largest' to arr[0], and then iterates through the array to find the maximum value. The output '5' is displayed at the bottom of the window.

```
main.py
1  arr = [1, 2, 3, 4, 5]
2
3  largest = arr[0]
4
5  for i in arr:
6      if i > largest:
7          largest = i
8
9  print(largest)

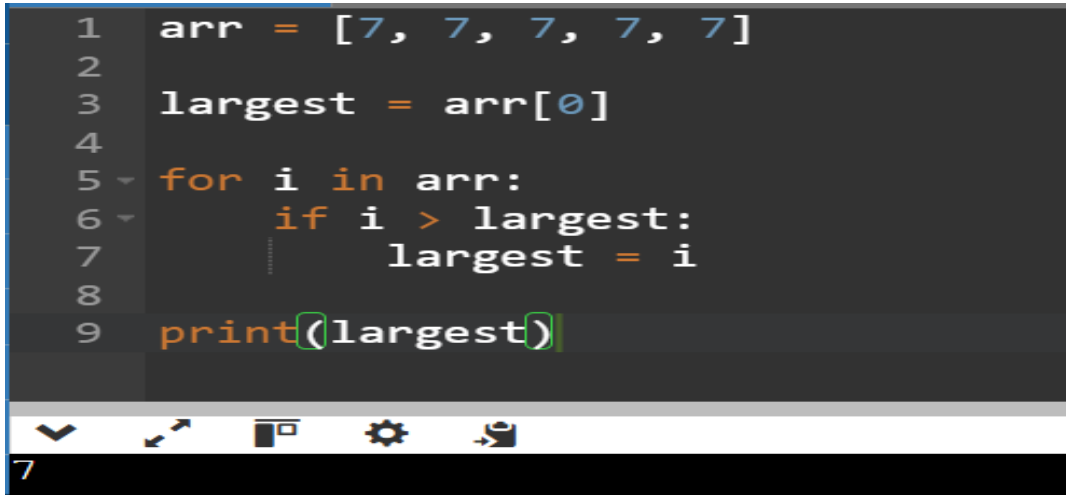
5
```

Time Complexity : $O(n)$

Space Complexity : $O(1)$

TEST:2

```
1 arr = [7, 7, 7, 7, 7]
2
3 largest = arr[0]
4
5 for i in arr:
6     if i > largest:
7         largest = i
8
9 print(largest)
```



Time Complexity

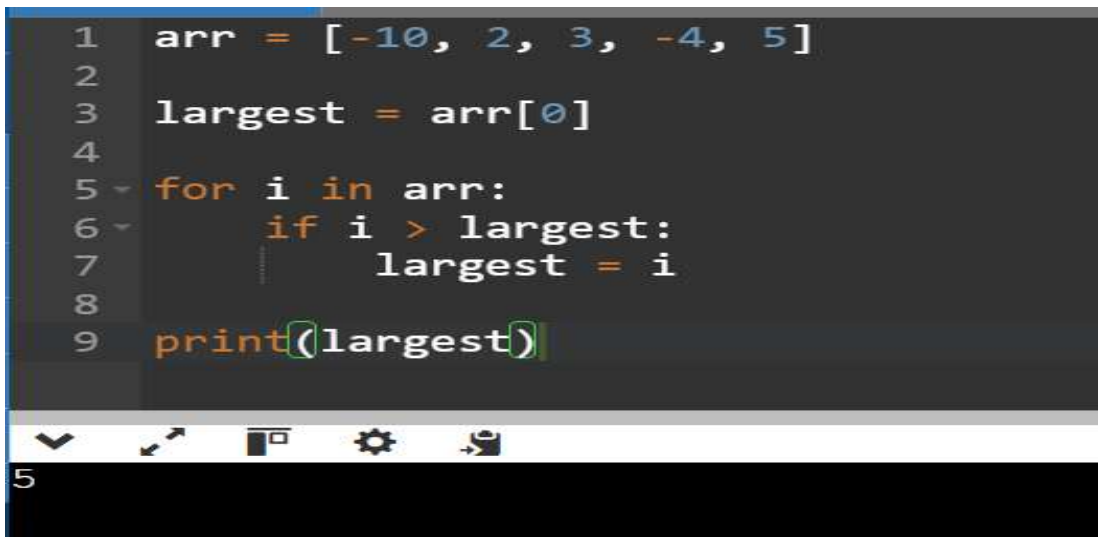
$O(n)$

Space Complexity

$O(1)$

TEST: 3

```
1 arr = [-10, 2, 3, -4, 5]
2
3 largest = arr[0]
4
5 for i in arr:
6     if i > largest:
7         largest = i
8
9 print(largest)
```



Time Complexity

$O(n)$

Space Complexity

$O(1)$

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

Test Cases

1. Empty List

1. Input: []

2. Expected Output: None or an appropriate message indicating that the list is empty.

2. Single Element List

1. Input: [5]

2. Expected Output: 5

3. All Elements are the Same

1. Input: [3, 3, 3, 3, 3]

2. Expected Output: 3

Aim

To write a Python program to **sort a list using an efficient sorting algorithm** and then **find the maximum element** in the sorted list.

Algorithm

1. Start.
2. Read the list.
3. Check if the list is empty.
4. If the list is empty, print "List is empty" and stop.
5. Sort the list using the `sort()` method.
6. The last element of the sorted list is the maximum element.
7. Print the maximum element.
8. Stop.

TEST:1

```
main.py
1  nums = []
2
3  if len(nums) == 0:
4      print("List is empty")
5  else:
6      nums.sort()
7      print(nums[-1])
```

List is empty

Time Complexity

$O(1)$

Space Complexity

$O(1)$

TEST:2

```
main.py
1  nums = [3, 3, 3, 3, 3]
2
3  if len(nums) == 0:
4      print("List is empty")
5  else:
6      nums.sort()
7      print(nums[-1])
```

3

Time Complexity

$O(n \log n)$

Space Complexity

$O(1)$

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases

Some Duplicate Elements

Input: [3, 7, 3, 5, 2, 5, 9, 2]

Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Negative and Positive Numbers

Input: [-1, 2, -1, 3, 2, -2]

Expected Output: [-1, 2, 3, -2] (Order may vary)

List with Large Numbers

Input: [1000000, 999999, 1000000]

Expected Output: [1000000, 999999]

Aim

To write a Python program to create a new list containing only the **unique elements** from the given list.

Algorithm

1. Start.
2. Read the input list.
3. Create an empty list unique.
4. Traverse each element in the input list.
5. If the element is not present in unique, add it.
6. Print the unique list.
7. Stop.

TEST:1

```
main.py
1  nums = [3, 7, 3, 5, 2, 5, 9, 2]
2
3  unique = []
4
5  for i in nums:
6      if i not in unique:
7          unique.append(i)
8
9  print(unique)
```

[3, 7, 5, 2, 9]

Time Complexity

$O(n^2)$

Space Complexity

$O(n)$

TEST:2

```
main.py
1  nums = [-1, 2, -1, 3, 2, -2]
2
3  unique = []
4
5  for i in nums:
6      if i not in unique:
7          unique.append(i)
8
9  print(unique)
```

[-1, 2, 3, -2]

Time Complexity

$O(n^2)$

Space Complexity

$O(n)$

8. Sort an array of integers using the bubble sort technique. Analyze its time

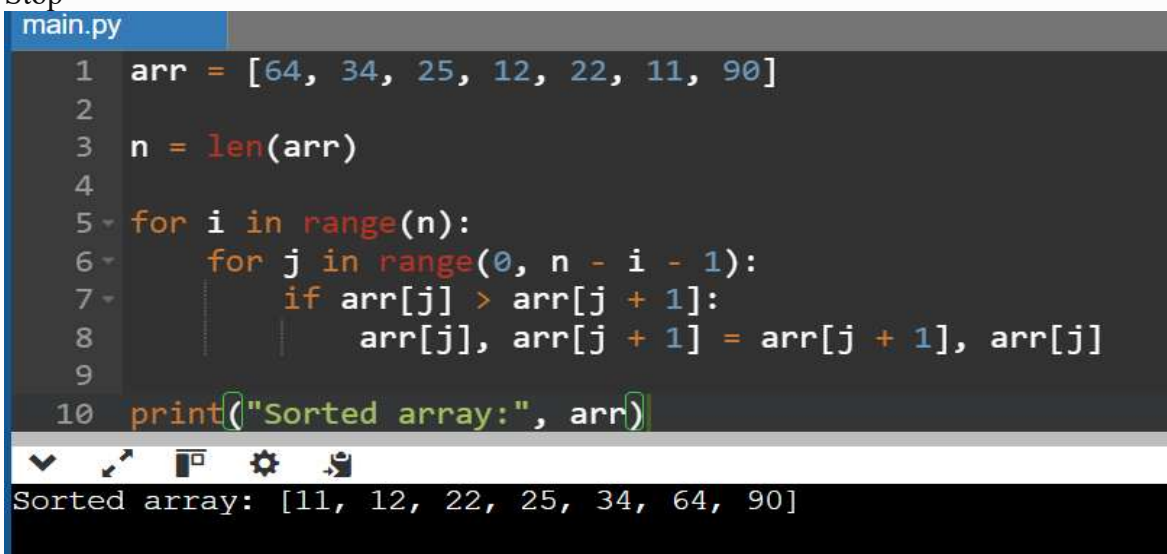
complexity using Big-O notation. Write the code

Aim

To write a Python program to sort an array of integers using the **Bubble Sort** technique.

Algorithm

1. Start.
2. Read the array.
3. Compare each adjacent pair of elements.
4. If the left element is greater than the right element, swap them.
5. Repeat the process for all elements until the array is sorted.
6. Print the sorted array.
7. Stop



```
main.py
1 arr = [64, 34, 25, 12, 22, 11, 90]
2
3 n = len(arr)
4
5 for i in range(n):
6     for j in range(0, n - i - 1):
7         if arr[j] > arr[j + 1]:
8             arr[j], arr[j + 1] = arr[j + 1], arr[j]
9
10 print("Sorted array:", arr)
```

Sorted array: [11, 12, 22, 25, 34, 64, 90]

Time Complexity

- **Best Case: $O(n)$** (when the array is already sorted and an optimized Bubble Sort is used)
- **Average Case: $O(n^2)$**
- **Worst Case: $O(n^2)$**

Space Complexity

$O(1)$ (In-place sorting; no extra memory is used.)

9. Checks if a given number x exists in a sorted array arr using binary search.

Analyze its time complexity using Big-O notation.

Test Case:

Example X={ 3,4,6,-9,10,8,9,30} KEY=10

Output: Element 10 is found at position 5

Example X={ 3,4,6,-9,10,8,9,30} KEY=100

Output : Element 100 is not found

Aim

To write a Python program to search for a given element in a **sorted array** using the **Binary Search** technique.

Algorithm

1. Start.
2. Read the array and the key.
3. Sort the array.
4. Initialize low = 0 and high = n - 1.
5. Repeat while low <= high:
 - o Find mid = (low + high) // 2.
 - o If arr[mid] == key, print the position and stop.
 - o If key < arr[mid], search the left half.
 - o Otherwise, search the right half.
6. If the element is not found, print "Element not found".
7. Stop.

```
1 arr = [3, 4, 6, -9, 10, 8, 9, 30]
2 key = 10
3
4 arr.sort()
5
6 low = 0
7 high = len(arr) - 1
8
9 while low <= high:
10     mid = (low + high) // 2
11
12     if arr[mid] == key:
13         print("Element", key, "is found at position", mid + 1)
14         break
15     elif key < arr[mid]:
16         high = mid - 1
17     else:
18         low = mid + 1
19 else:
20     print("Element", key, "is not found")
```

input

Element 10 is found at position 7

Example 2

```
main.py
1 arr = [3, 4, 6, -9, 10, 8, 9, 30]
2 key = 100
3
4 arr.sort()
5
6 low = 0
7 high = len(arr) - 1
8
9 while low <= high:
10     mid = (low + high) // 2
11
12     if arr[mid] == key:
13         print("Element", key, "is found at position", mid + 1)
14         break
15     elif key < arr[mid]:
16         high = mid - 1
17     else:
18         low = mid + 1
19 else:
20     print("Element", key, "is not found")
```

input

Element 100 is not found

Time Complexity

- Best Case: $O(1)$
- Average Case: $O(\log n)$
- Worst Case: $O(\log n)$

Space Complexity

$O(1)$

10. Given an array of integers `nums`, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

Aim

To write a Python program to sort an array in ascending order using **Merge Sort** without using any built-in sorting functions.

Algorithm

1. Start.
2. Divide the array into two halves.
3. Recursively sort the left half.
4. Recursively sort the right half.
5. Merge the two sorted halves.
6. Repeat until the entire array is sorted.
7. Print the sorted array.
8. Stop.



```
main.py
1- def mergeSort(arr):
2-     if len(arr) > 1:
3-
4-         mid = len(arr) // 2
5-         left = arr[:mid]
6-         right = arr[mid:]
7-
8-         mergeSort(left)
9-         mergeSort(right)
10-
11-         i = j = k = 0
12-
13-         while i < len(left) and j < len(right):
14-             if left[i] < right[j]:
15-                 arr[k] = left[i]
16-                 i += 1
17-             else:
18-                 arr[k] = right[j]
19-                 j += 1
20-             k += 1
21-
22-         while i < len(left):
23-             arr[k] = left[i]
24-             i += 1
25-
26-         while j < len(right):
27-             arr[k] = right[j]
28-             j += 1
29-             k += 1
30-
31-         return arr
32-
33- arr = [5, 2, 3, 1]
34- mergeSort(arr)
35- print("Sorted Array: ", arr)
```

Sorted Array: [1, 2, 3, 5]

Time Complexity

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n \log n)$

Space Complexity

$O(n)$ (Auxiliary space used during merging)

11. Given an $m \times n$ grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

Example 1

Input: $m = 2, n = 2, N = 2, i = 0, j = 0$

Output: 6

Example 2

Input: $m = 1, n = 3, N = 3, i = 0, j = 1$

Output: 12

Aim

To write a Python program to find the number of ways to move a ball out of the grid boundary in exactly N steps.

Algorithm

1. Start.
2. Read the values of m, n, N, i , and j .
3. Define a recursive function with memoization.
4. If the ball moves outside the grid, return 1.
5. If no steps are remaining and the ball is still inside the grid, return 0.
6. Recursively move the ball in four directions (up, down, left, right).
7. Store previously computed results to avoid repeated calculations.
8. Print the total number of paths.
9. Stop.

Example 1

```
1 from functools import lru_cache
2
3 def findPaths(m, n, N, i, j):
4     @lru_cache(None)
5     def dfs(x, y, steps):
6         if x < 0 or x >= m or y < 0 or y >= n:
7             return 1
8         if steps == 0:
9             return 0
10        return (
11            dfs(x + 1, y, steps - 1) +
12            dfs(x - 1, y, steps - 1) +
13            dfs(x, y + 1, steps - 1) +
14            dfs(x, y - 1, steps - 1)
15        )
16    return dfs(i, j, N)
17
18 m = 2
19 n = 2
```

6

Example 2

```
1 from functools import lru_cache
2 def findPaths(m, n, N, i, j):
3     @lru_cache(None)
4     def dfs(x, y, steps):
5         if x < 0 or x >= m or y < 0 or y >= n:
6             return 1
7         if steps == 0:
8             return 0
9         return (
10            dfs(x + 1, y, steps - 1) +
11            dfs(x - 1, y, steps - 1) +
12            dfs(x, y + 1, steps - 1) +
13            dfs(x, y - 1, steps - 1)
14        )
15    return dfs(i, j, N)
16 m = 1
17 n = 3
18 N = 3
19 i = 0
20 j = 1
21
22 print(findPaths(m, n, N, i, j))
```

12

Time Complexity

$O(m \times n \times N)$

Space Complexity

$O(m \times n \times N)$

12. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Example 1

Input: nums = [2, 3, 2]

Output: 3

Explanation: The maximum money you can rob without alerting the police is 3 (robbing house 2).

Example 2

Input: nums = [1, 2, 3, 1]

Output: 4

Explanation: The maximum money you can rob without alerting the police is 4 (robbing houses 1 and 3).

Aim

To write a Python program to find the maximum amount of money that can be robbed from circularly arranged houses without robbing two adjacent houses.

Algorithm

1. Start.
2. Read the array nums.
3. If there is only one house, return its value.
4. Since the houses are in a circle, solve two cases:
 - Exclude the first house.
 - Exclude the last house.
5. For each case, use dynamic programming to find the maximum money.
6. Return the maximum of the two results.
7. Stop.

Example : 1

```
main.py
1 - def robLinear(nums):
2     prev1 = 0
3     prev2 = 0
4
5     for money in nums:
6         temp = max(prev1, prev2 + money)
7         prev2 = prev1
8         prev1 = temp
9
10    return prev1
11
12
13 - def rob(nums):
14     if len(nums) == 1:
15         return nums[0]
16
17     return max(robLinear(nums[:-1]), robLinear(nums[1:]))
18
19 nums = [2, 3, 2]
20
21 print(rob(nums))
```

input

3

Example: 2

```
1 - def robLinear(nums):
2     prev1 = 0
3     prev2 = 0
4
5     for money in nums:
6         temp = max(prev1, prev2 + money)
7         prev2 = prev1
8         prev1 = temp
9
10    return prev1
11
12
13 - def rob(nums):
14     if len(nums) == 1:
15         return nums[0]
16
17     return max(robLinear(nums[:-1]), robLinear(nums[1:]))
18
19 nums = [1, 2, 3, 1]
20
21 print(rob(nums))
```

input

4

Time Complexity

$O(n)$

Space Complexity

$O(1)$

13. You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1

Input: $n = 4$

Output: 5

Example 2

Input: $n = 3$

Output: 3

Aim

To write a Python program to find the number of distinct ways to climb a staircase when you can climb either **1 step** or **2 steps** at a time.

Algorithm

1. Start.
2. Read the value of n .
3. If n is 1, return 1.
4. Initialize two variables:
 - $first = 1$
 - $second = 2$
5. Traverse from step 3 to n .
6. Compute the current number of ways as the sum of the previous two values.
7. Update the variables.
8. Print the total number of ways.
9. Stop.

Example:1

```
1 - def climbStairs(n):
2 -     if n == 1:
3 -         return 1
4 -     if n == 2:
5 -         return 2
6 -
7 -     first = 1
8 -     second = 2
9 -
10 -    for i in range(3, n + 1):
11 -        third = first + second
12 -        first = second
13 -        second = third
14 -
15 -    return second
16 -
17 - n = 4
18 - print(climbStairs(n))
```

5

Example: 2

```
1 - def climbStairs(n):
2 -     if n == 1:
3 -         return 1
4 -     if n == 2:
5 -         return 2
6 -
7 -     first = 1
8 -     second = 2
9 -
10 -    for i in range(3, n + 1):
11 -        third = first + second
12 -        first = second
13 -        second = third
14 -
15 -    return second
16 -
17 - n = 3
18 - print(climbStairs(n))
```

3

Time Complexity

$O(n)$

Space Complexity

$O(1)$

14. A robot is located at the top-left corner of a $m \times n$ grid. The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Example 1

Input: $m = 7, n = 3$

Output: 28

Example 2

Input: $m = 3, n = 2$

Output: 3

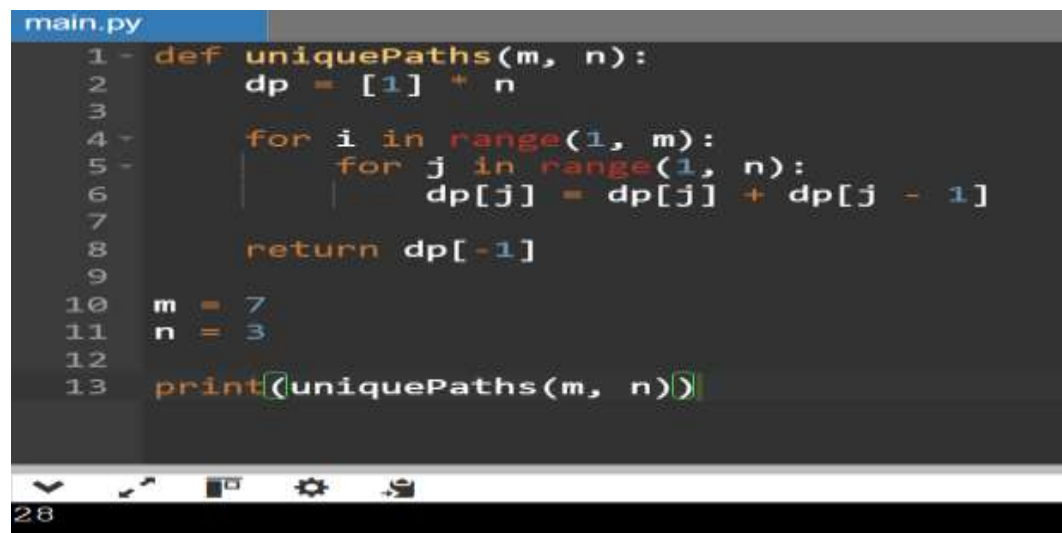
Aim

To write a Python program to find the number of **unique paths** for a robot to move from the top-left corner to the bottom-right corner of an $m \times n$ grid

Algorithm

1. Start.
2. Read the values of m and n .
3. Create a one-dimensional array of size n and initialize all elements to 1.
4. Traverse the rows from 1 to $m-1$.
5. For each row, update the number of paths by adding the left cell value.
6. The last element of the array gives the total number of unique paths.
7. Print the result.
8. Stop.

Example;1

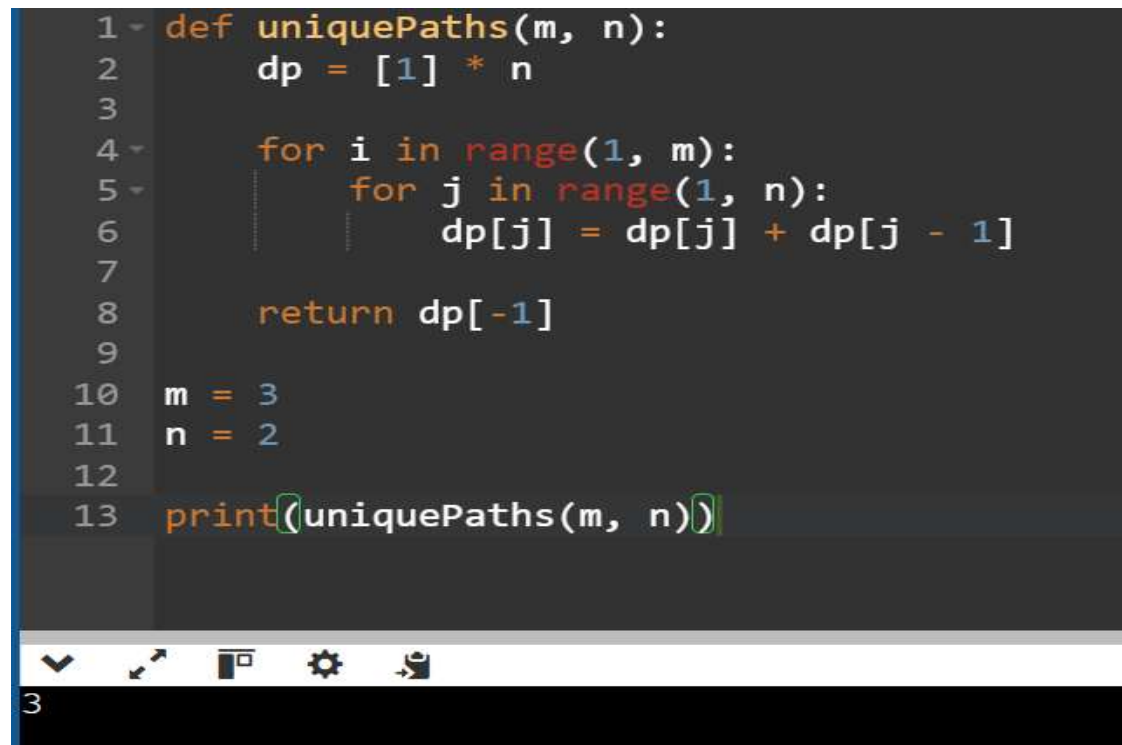


```
main.py
1 - def uniquePaths(m, n):
2     dp = [1] * n
3
4     for i in range(1, m):
5         for j in range(1, n):
6             dp[j] = dp[j] + dp[j - 1]
7
8     return dp[-1]
9
10 m = 7
11 n = 3
12
13 print(uniquePaths(m, n))
```

28

Example: 2

```
1 def uniquePaths(m, n):
2     dp = [1] * n
3
4     for i in range(1, m):
5         for j in range(1, n):
6             dp[j] = dp[j] + dp[j - 1]
7
8     return dp[-1]
9
10 m = 3
11 n = 2
12
13 print(uniquePaths(m, n))
```

A screenshot of a code editor with a dark background. The code is written in Python. It defines a function uniquePaths(m, n) that uses a dynamic programming array dp of size n. The array is initialized with 1s. Then, for each row i from 1 to m-1, and for each column j from 1 to n-1, the value at dp[j] is updated to be the sum of its current value and the value at dp[j-1]. Finally, the value at dp[-1] is returned. Below the function definition, the variables m and n are set to 3 and 2 respectively, and the function is called with these values. The output of the function is printed, which is 3. The code editor has a toolbar at the bottom with icons for undo, redo, search, and other editing functions.

Time Complexity

$O(m \times n)$

Space Complexity

$O(n)$

15. In a string S of lowercase letters, these letters form consecutive groups of the same character. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.

Example 1

Input: s = "abbxxxxzzy"

Output: [[3,6]]

Explanation: "xxxx" is the only large group with start index 3 and end index 6.

Example 2

Input: s = "abc"

Output: []

Explanation: There are no large groups.

Aim

To write a Python program to find the **start and end indices of all large groups** (groups with 3 or more consecutive identical characters) in a string.

Algorithm

1. Start.
2. Read the input string s.
3. Initialize an empty list result.
4. Traverse the string and count consecutive identical characters.
5. If a group has length greater than or equal to 3, store its start and end indices.
6. Continue until the end of the string.
7. Print the list of intervals.
8. Stop.

Example: 1

```
1 def largeGroupPositions(s):
2     result = []
3     start = 0
4
5     for i in range(1, len(s) + 1):
6         if i == len(s) or s[i] != s[start]:
7             if i - start >= 3:
8                 result.append([start, i - 1])
9                 start = i
10
11     return result
12
13 s = "abbxxxxzzy"
14 print(largeGroupPositions(s))
```

[[3, 6]]

Example: 2

```
1 def largeGroupPositions(s):
2     result = []
3     start = 0
4
5     for i in range(1, len(s) + 1):
6         if i == len(s) or s[i] != s[start]:
7             if i - start >= 3:
8                 result.append([start, i - 1])
9                 start = i
10
11     return result
12
13 s = "abc"
14 print(largeGroupPositions(s))
```

[]

Time Complexity

$O(n)$

Space Complexity

$O(1)$ (excluding the output list)

15. The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970. Given the current state of the $m \times n$ grid board, return the next state according to the Game of Life rules.

Example 1

Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]

Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]

Example 2

Input: board = [[1,1],[1,0]]

Output: [[1,1],[1,1]]

Aim

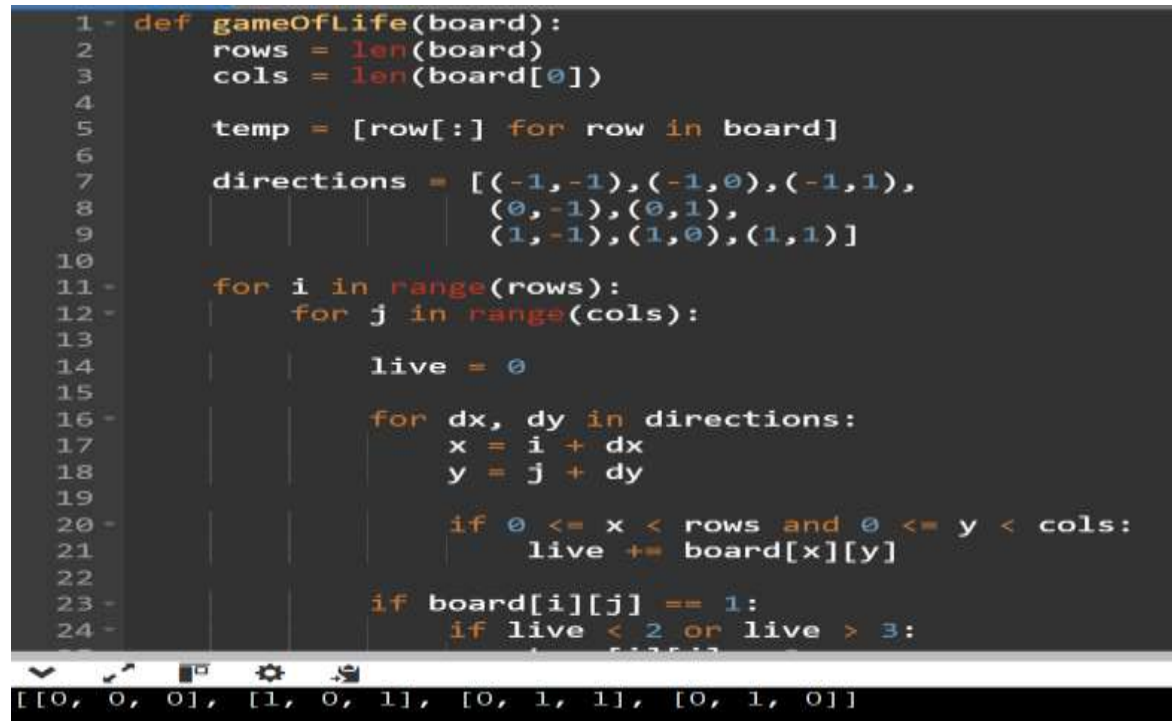
To write a Python program to compute the **next state** of the Game of Life board based on Conway's rules.

Algorithm

1. Start.
2. Read the input board.
3. Create a copy of the board.
4. For each cell, count its eight live neighbors.
5. Apply the Game of Life rules:
 - A live cell with fewer than 2 or more than 3 live neighbors dies.
 - A live cell with 2 or 3 live neighbors lives.
 - A dead cell with exactly 3 live neighbors becomes alive.
6. Update the board.
7. Print the new board.
8. Stop.

Example 1

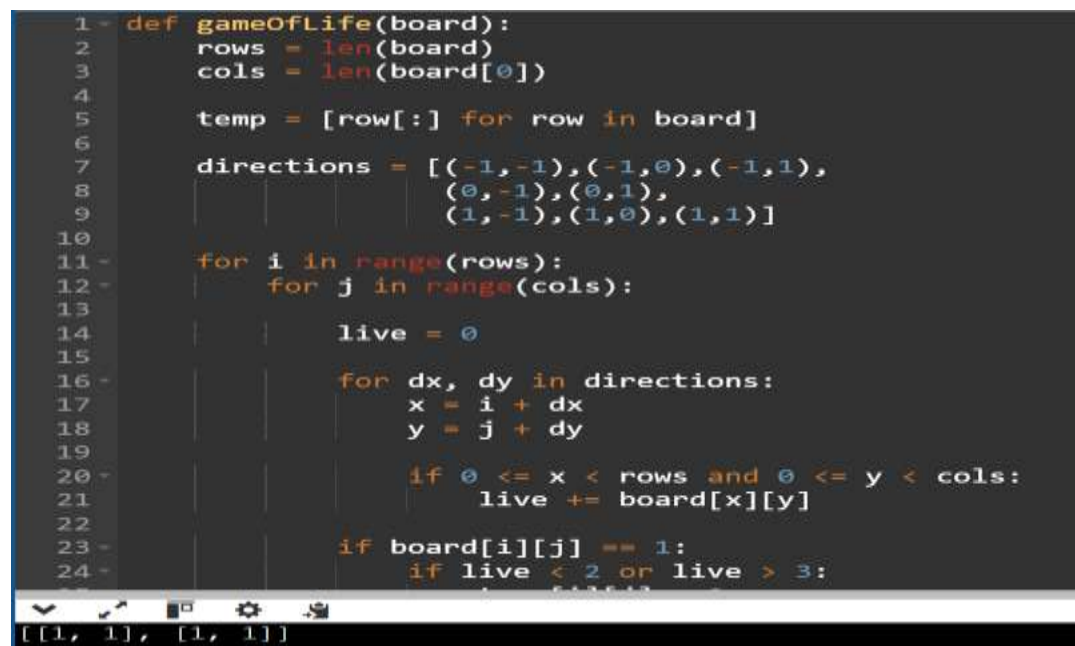
```
1- def gameOfLife(board):
2-     rows = len(board)
3-     cols = len(board[0])
4-
5-     temp = [row[:] for row in board]
6-
7-     directions = [(-1,-1),(-1,0),(-1,1),
8-                  (0,-1),(0,1),
9-                  (1,-1),(1,0),(1,1)]
10-
11-     for i in range(rows):
12-         for j in range(cols):
13-
14-             live = 0
15-
16-             for dx, dy in directions:
17-                 x = i + dx
18-                 y = j + dy
19-
20-                 if 0 <= x < rows and 0 <= y < cols:
21-                     live += board[x][y]
22-
23-             if board[i][j] == 1:
24-                 if live < 2 or live > 3:
```



```
[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
```

Example 2

```
1- def gameOfLife(board):
2-     rows = len(board)
3-     cols = len(board[0])
4-
5-     temp = [row[:] for row in board]
6-
7-     directions = [(-1,-1),(-1,0),(-1,1),
8-                  (0,-1),(0,1),
9-                  (1,-1),(1,0),(1,1)]
10-
11-     for i in range(rows):
12-         for j in range(cols):
13-
14-             live = 0
15-
16-             for dx, dy in directions:
17-                 x = i + dx
18-                 y = j + dy
19-
20-                 if 0 <= x < rows and 0 <= y < cols:
21-                     live += board[x][y]
22-
23-             if board[i][j] == 1:
24-                 if live < 2 or live > 3:
```



```
[[1, 1], [1, 1]]
```

Time Complexity

$O(m \times n)$

Space Complexity

$O(m \times n)$

16. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on. Each glass holds one cup of champagne. When a glass overflows, the excess is equally divided between the two glasses below it. Given the amount of champagne poured, return how full the query_glass in the query_row is.

Example 1

Input: poured = 1, query_row = 1, query_glass = 1

Output: 0.00000

Example 2

Input: poured = 2, query_row = 1, query_glass = 1

Output: 0.50000

Aim

To write a Python program to determine how full a particular glass is in a champagne tower after pouring a given number of cups.

Algorithm

1. Start.
2. Read poured, query_row, and query_glass.
3. Create a 2D array to represent the champagne tower.
4. Pour all the champagne into the top glass.
5. For each row, if a glass contains more than one cup, distribute the excess equally to the two glasses below.
6. Continue until the required row is processed.
7. Return the minimum of 1 and the amount in the required glass.
8. Stop.

Example: 1

```
1- def champagneTower(poured, query_row, query_glass):
2-     tower = [[0.0] * 101 for _ in range(101)]
3-     tower[0][0] = poured
4-
5-     for row in range(query_row + 1):
6-         for glass in range(row + 1):
7-             excess = (tower[row][glass] - 1.0) / 2.0
8-
9-             if excess > 0:
10-                 tower[row + 1][glass] += excess
11-                 tower[row + 1][glass + 1] += excess
12-
13-     return min(1.0, tower[query_row][query_glass])
14-
15-
16- poured = 1
17- query_row = 1
18- query_glass = 1
19-
20- print("{:.5f}".format(champagneTower(poured, query_row, query_glass)))
```

input

0.00000

Time Complexity : $O(\text{query_row}^2)$

Space Complexity : $O(\text{query_row}^2)$

Example:2

```
1- def champagneTower(poured, query_row, query_glass):
2-     tower = [[0.0] * 101 for _ in range(101)]
3-     tower[0][0] = poured
4-
5-     for row in range(query_row + 1):
6-         for glass in range(row + 1):
7-             excess = (tower[row][glass] - 1.0) / 2.0
8-
9-             if excess > 0:
10-                 tower[row + 1][glass] += excess
11-                 tower[row + 1][glass + 1] += excess
12-
13-     return min(1.0, tower[query_row][query_glass])
14-
15-
16- poured = 2
17- query_row = 1
18- query_glass = 1
19-
20- print("{:.5f}".format(champagneTower(poured, query_row, query_glass)))
```

input

0.50000

Time Complexity

$O(\text{query_row}^2)$

Space Complexity

$O(\text{query_row}^2)$